



Carrera:
Programación y WebMaster

Docente:
Mario Alonso Segovia Gutierrez

Materia:
Proyecto de una Plataforma Virtual

Alumno(s):
Bryant Maximiliano Dzul Perez

Fecha:
20 de Julio 2026

Introducción

A continuación se exponen los resultados de la auditoría de calidad realizada a la plataforma web GymManager, un sistema diseñado para la administración de un gimnasio. A pesar de que el proyecto lleva ocho meses de desarrollo, han comenzado a presentar retrasos significativos y un aumento considerable en los costos de mantenimiento. El objetivo de este documento es analizar el estado actual del código y la gestión del proyecto para identificar malas prácticas, evaluar el incumplimiento de principios arquitectónicos como SOLID y Clean Code, y proponer un plan de acción riguroso que incluya la definición de un Producto Mínimo Viable (MVP) y una *Definition of Done* (DoD) formal para reconducir el desarrollo hacia estándares de alta calidad.

Identificación de problemas

1. Clases de más de 1,800 líneas de código.

¿Qué ocurre? Existen clases con miles de líneas de código en el sistema.

Mala práctica: Esto provoca que una sola clase concentra demasiada información y comportamiento.

Consecuencias: Hace que el código sea ilegible, difícil de testear y extremadamente propenso a errores al intentar modificar una pequeña parte, dificultando el mantenimiento.

2. Funciones de más de 250 líneas.

¿Qué ocurre? Se encontraron métodos y funciones excesivamente largos.

Mala práctica: Viola el principio de que las funciones deben ser pequeñas y hacer una sola cosa.

Consecuencias: Dificulta el rastreo de errores, impide la reutilización de código lógico y complica la lectura para nuevos desarrolladores.

3. Código duplicado por copiar y pegar.

¿Qué ocurre? Se reutiliza código copiando y pegando fragmentos idénticos entre diferentes módulos.

Mala práctica: Viola el principio DRY (Don't Repeat Yourself).

Consecuencias: Si se descubre un error en ese fragmento, deberá buscarse y corregirse manualmente en múltiples lugares, generando inconsistencias y bugs.

4. Nomenclatura no descriptiva (x, dato, temp, valor1, obj).

¿Qué ocurre? Las variables no revelan su intención ni el tipo de dato que almacenan.

Mala práctica: Falta de nombres significativos y pronunciables.

Consecuencias: Obliga a los programadores a descifrar el contexto y el propósito de cada variable leyendo toda la función, aumentando dramáticamente el tiempo de comprensión.

5. Centralización de responsabilidades en un solo archivo.

¿Qué ocurre? Un solo archivo valida datos, consulta la base de datos, genera reportes y envía correos.

Mala práctica: Mal uso del Principio de Responsabilidad Única (SRP).

Consecuencias: Genera un alto nivel de acoplamiento. Un cambio en la lógica de correos o generación de reportes podría romper las consultas a la base de datos.

6. Inconsistencia en la estructura del proyecto.

¿Qué ocurre? Cada desarrollador organiza sus archivos y carpetas de manera distinta.

Mala práctica: Falta de arquitectura de software y estandarización del proyecto.

Consecuencias: Desorden estructural que ralentiza la navegación del código y genera conflictos al integrar el trabajo de varios desarrolladores.

7. Ausencia de guía de estilos.

¿Qué ocurre? No existe una convención estandarizada sobre cómo escribir y formatear el código.

Mala práctica: No existe una identidad del sitio que estandarice los colores a utilizar.

Consecuencias: Aumenta la carga cognitiva del equipo al leer el código y dificulta la detección de errores de sintaxis.

8. Comentarios redundantes.

¿Qué ocurre? Los comentarios se dedican a explicar lo que hace cada línea en vez de explicar el "por qué" o usar código claro.

Mala práctica: Usar comentarios como muletas para justificar código incomprensible.

Consecuencias: Ruido visual masivo y comentarios que rápidamente quedarán desactualizados cuando el código cambie, volviéndose mentiras que confunden al equipo.

9. Criterio de finalización subjetivo.

¿Qué ocurre? Las tareas se dan por terminadas cuando "parecen funcionar".

Mala práctica: Ausencia de pruebas automatizadas y de criterios formales de aceptación.

Consecuencias: Alta probabilidad de entregar software con defectos críticos a producción y fallos en casos límite no contemplados.

10. Falta de revisiones de código.

¿Qué ocurre? Nunca se hace revisión entre compañeros.

Mala práctica: Falta de aseguramiento de calidad colaborativo.

Consecuencias: Se perpetúan las malas prácticas, se filtran vulnerabilidades y se pierde la oportunidad de compartir conocimiento sobre el sistema dentro del equipo.

11. Alto acoplamiento en funcionalidades.

¿Qué ocurre? Agregar una función nueva requiere modificar múltiples clases que ya existían.

Mala práctica: Violación del principio Abierto/Cerrado y alta dependencia entre módulos.

Consecuencias: El sistema se vuelve rígido y frágil; las actualizaciones son lentas y peligrosas por el efecto dominó de los cambios.

12. Desarrollo de funcionalidades no solicitadas.

¿Qué ocurre? Se programaron agregados porque el equipo lo consideró sin petición del cliente.

Mala práctica: Falta de apego a los requerimientos y sobreingeniería.

Consecuencias: Desperdicio de tiempo, presupuesto y esfuerzo, además de inyectar mayor complejidad y potenciales bugs al sistema sin aportar valor real.

Análisis de Clean Code

1. Nombres de variables poco descriptivos (x, dato, temp).

Principio incumplido: *Intention-Revealing Names* (Nombres que revelen la intención).

Corrección: Renombrar las variables según su dominio de negocio. Por ejemplo, cambiar `temp` por `diasRestantesMembresia`, o `obj` por `entrenadorActual`.

Beneficios: El código se vuelve autodescriptivo. Cualquier desarrollador que lo lea sabrá exactamente qué flujo de datos se está procesando sin necesidad de buscar el origen de la variable.

2. Funciones de más de 250 líneas.

Principio incumplido: *Small Functions / Do One Thing* (Las funciones deben ser pequeñas y hacer una sola cosa).

Corrección: Aplicar la técnica de refactorización "Extraer Método". Dividir la función enorme en funciones privadas más pequeñas donde cada una realice un paso lógico específico.

Beneficios: Facilita la implementación de pruebas unitarias y permite la reutilización de bloques lógicos sin duplicar código.

3. Código duplicado copiado y pegado.

Principio incumplido: *DRY (Don't Repeat Yourself)*.

Corrección: Identificar las líneas repetidas, abstraerse en un método o clase de utilidad compartida, e invocar esa nueva función desde los distintos módulos que la requieran.

Beneficios: Un único punto de mantenimiento. Si cambia la regla de negocio, solo se modifica un método y el cambio se refleja en todo el sistema.

4. Comentarios que explican cada línea.

Principio incumplido: *Explain Yourself in Code* (El código debe explicarse a sí mismo). Los comentarios deben usarse para el "por qué", no para el "qué".

Corrección: Borrar los comentarios y refactorizar el código mal escrito utilizando variables explicativas y métodos bien nombrados que hagan evidente la acción.

Beneficios: Se elimina el mantenimiento de los comentarios, se reduce la deuda técnica generada por comentarios desactualizados y se fuerza al programador a escribir un código inherentemente más legible.

5. Cada desarrollador utiliza una estructura y estilo diferente.

Principio incumplido: *Formatting and Team Rules* (Formateo y reglas de equipo).

Corrección: Definir una arquitectura estándar y configurar herramientas de análisis estático en el editor de código y repositorios para forzar reglas uniformes antes de guardar.

Beneficios: Todo el código parece escrito por una sola persona. Esto disminuye la fatiga mental y acelera la integración de nuevos miembros al equipo.

Aplicación de los principios SOLID

1. Single Responsibility Principle (SRP) - Principio de Responsabilidad Única

- **En qué consiste:** Una clase o módulo debe tener una y solo una razón para cambiar; es decir, debe encargarse de una única responsabilidad dentro del sistema.
- **Evidencia en el caso:** Un mismo archivo realiza validaciones, hace consultas a la base de datos, genera reportes y envía correos electrónicos.
- **Cambios a realizar:** Desacoplar el archivo aplicando patrones de diseño. Crear una clase para la conexión/consultas (Patrón Repository), una clase para los servicios de envío de correo, una clase separada para las validaciones de entrada (Requests) y un controlador que orqueste estos servicios separados.
- **Beneficios:** Alta cohesión. Si cambia el proveedor de correos electrónicos, solo se modificará la clase de *EmailService*, garantizando que los reportes y las bases de datos no se vean afectados por el cambio.

2. Open/Closed Principle (OCP) - Principio Abierto/Cerrado

- **En qué consiste:** Las entidades de software deben estar abiertas para su extensión, pero cerradas para su modificación. Se debe poder agregar nueva funcionalidad sin alterar el código existente que ya funciona.
- **Evidencia en el caso:** Cada nueva funcionalidad requiere modificar varias clases existentes.
- **Cambios a realizar:** Implementar polimorfismo mediante el uso de interfaces o clases abstractas. En lugar de tener sentencias **if/switch** masivas que deban editarse cada vez que hay un caso nuevo, se inyectan nuevas dependencias que cumplan con la interfaz establecida.
- **Beneficios:** Minimiza enormemente el riesgo de romper funcionalidades existentes al agregar módulos nuevos, haciendo que el sistema sea escalable y robusto.

Análisis de la deuda técnica

Decisiones que generan deuda técnica:

1. No corregir errores conocidos para "avanzar más rápido".
2. Implementar funcionalidades no solicitadas por el cliente (*Gold Plating*).
3. No actualizar la documentación del proyecto.
4. Integrar código sin revisiones ni validación formal.

Tipos de deuda técnica observada:

1. **Deuda Deliberada/Prudente (pero mal gestionada):** Dejar errores conocidos temporalmente para avanzar.
2. **Deuda Inadvertida/Imprudente:** La ausencia de convenciones, acoplamiento extremo, clases masivas e ignorancia arquitectónica reflejan desconocimiento y falta de liderazgo técnico.

Afectación al mantenimiento: El código se ha vuelto un sistema legado a los 8 meses. Nadie querrá ni podrá tocar el código sin miedo a romper otra parte del sistema.

Impacto en tiempo y costo: Los tiempos de desarrollo aumentarán exponencialmente porque entender el espagueti de código y corregir los efectos secundarios tomará más tiempo que programar en sí. Los costos de mantenimiento, que ya han aumentado, seguirán disparándose al requerir más horas-hombre.

Acciones propuestas para disminuirla:

1. Frenar temporalmente el desarrollo de nuevas características (Feature Freeze).
2. Aplicar un proceso sistemático de refactorización progresiva aplicando la "Regla del Boy Scout" (dejar el archivo más limpio de lo que se encontró).
3. Implementar *Code Reviews* obligatorios mediante *Pull Requests*.
4. Auditar y limpiar el proyecto de funcionalidades no solicitadas.

Definición del Producto Mínimo Viable (MVP)

Funcionalidades del MVP (Núcleo Operativo):

- Gestión de clientes.
- Registro de pagos.
- Control de asistencia.
- Reservación de clases.
- Reportes básicos de ingresos.

Justificación: Estas son las herramientas fundamentales que el dueño de un gimnasio necesita para que su negocio opere desde el principio. Sin estas características, el software no es útil como sistema administrativo. Desarrollarlas requiere menos tiempo y asegura el retorno de inversión inicial.

Funcionalidades para futuras versiones (Agregados de Valor):

- Chat interno entre entrenadores.
- Integración con relojes inteligentes.
- Sistema de recompensas.
- Publicación automática en redes sociales.
- Tienda en línea de suplementos.

Justificación: Representan una alta complejidad de desarrollo, integraciones con hardware y APIs de terceros. Retrasarían el lanzamiento. Son características de marketing y retención, que solo tienen sentido si el sistema base ya funciona perfectamente.

Definition of Done (DoD)

1. **Revisión de Código:** El código ha sido revisado, comentado y aprobado por al menos un desarrollador distinto al autor mediante un *Pull Request*.
2. **Guía de Estilos:** El código pasa exitosamente las herramientas de análisis estático sin emitir advertencias de formato o sintaxis.
3. **Modularidad:** Ninguna función nueva o modificada supera las 50 líneas de código y las responsabilidades están separadas (SRP).
4. **Manejo de Errores:** Todos los errores conocidos relacionados con la tarea han sido corregidos de raíz, no ocultos.
5. **Cero Duplicación:** La funcionalidad no contiene código copiado de otros módulos; utiliza implementaciones compartidas.
6. **Nomenclatura Limpia:** Las variables, funciones y clases tienen nombres descriptivos en idioma estandarizado.
7. **Fidelidad a los Requerimientos:** Se implementó estrictamente lo que se documentó en la historia de usuario.
8. **Funcionamiento Comprobado:** La funcionalidad ha sido probada de extremo a extremo, manejando correctamente casos de éxito.
9. **Documentación:** La documentación técnica interna y los manuales han sido actualizados con los cambios de arquitectura y flujo.
10. **Integración Limpia:** La rama de la característica se integra con la rama principal sin conflictos de código.

Conclusión

Considero que el tema más importante es la aplicación de los principios SOLID. Aunque el *Clean Code* garantiza que un código sea legible, los principios SOLID dictan la arquitectura a nivel de bloques constructivos. Sin SOLID, por muy bonitos que sean los nombres de las variables, el sistema colapsará bajo su propio peso.

En un proyecto real, ignorar estos principios desencadena exactamente lo que ocurrió en GymManager: sobrecostos, retrasos masivos, frustración en el equipo de desarrollo, rotación de personal y pérdida de confianza del cliente.

Utilizar estos principios me han permitido analizar y reestructurar proyectos que han llevado tiempo desarrollando, mejorándolos hasta el día de hoy o haciendo refactorización antes de que el proyecto escale demasiado y por supuesto integrando todo desde el día uno para reducir la deuda técnica.